



LECTURE 5

DEEP REPRESENTATION LEARNING II

How Can a Model Generalize?

PRESS **→** OR SPACE. 

© Marc Rußwurm

Licensed under [Creative Commons Attribution-NonCommercial 4.0 International \(CC BY-NC 4.0\)](#).

You may share and adapt these slides for teaching, research, and other non-commercial educational purposes with attribution.

Commercial training, paid workshops, consulting seminars, or incorporation into commercial course products require prior permission.

Learning Outcomes

- 1 Explain **generalization** as the difference between optimizing a finite training sample and performing well on the underlying data distribution.
- 2 Explain how **model capacity, data, and regularization** determine underfitting and overfitting in the classical regime.
- 3 Explain why the **interpolation threshold, double descent, and grokking** challenge the classical bias–variance picture and motivate the modern overparameterized learning regime.

We Know How to Minimize a Loss

WHAT WE HAVE FROM LECTURE 4

- Forward pass
- Loss computation
- Backpropagation
- Gradient-based parameter updates

We now know how to minimize training loss.

But is minimizing training loss the same as learning?

Is Imitation the Same as Learning?



What is wrong here?

Is this person learning — or only copying?

The Missing Ingredient: Generalization

WHAT WE HAVE

A model that fits the training examples — low training loss.

WHAT WE NEED

A model that performs well on situations it has not seen during training.

Fitting observed examples is not the same as learning something that transfers to new situations.

Generalization is the missing ingredient.

What Does a Learning System Need?

LECTURE 4

Model

architectures, parameters,
activations

LECTURE 5 ↓

Data & Experience

finite samples from a
distribution

LECTURE 4

Learning Algorithm

loss, gradients, backprop,
updates

LECTURE 5 ↓

Generalization

performance on unseen
situations

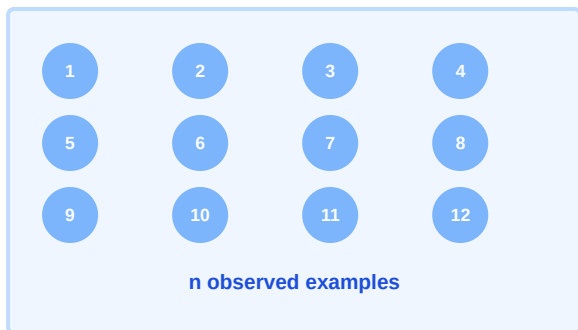
Lecture 4 covered the **model** and the **learning algorithm**. Lecture 5 focuses on **data** and **generalization**.

From finite training sets to expected loss — the classical view of generalization.

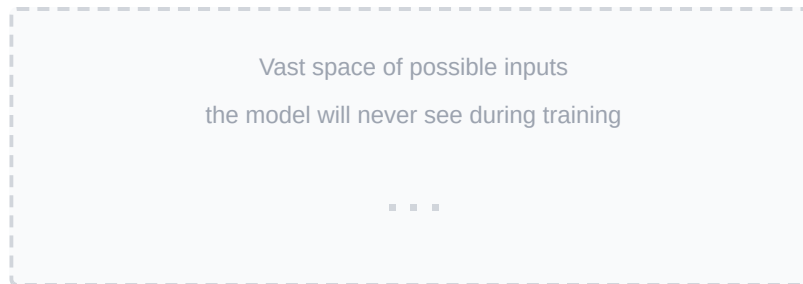
Block 1 — Classical Generalization

The Training Set Is Only a Sample

WHAT THE MODEL SEES



THE WORLD IT SHOULD LEARN TO HANDLE



The model only sees one finite sample — but must work on the rest.

Data Come From an Underlying Distribution

UNDERLYING DISTRIBUTION

$$p(x, y)$$

The true, unknown joint distribution
over inputs and targets

OBSERVED DATASET

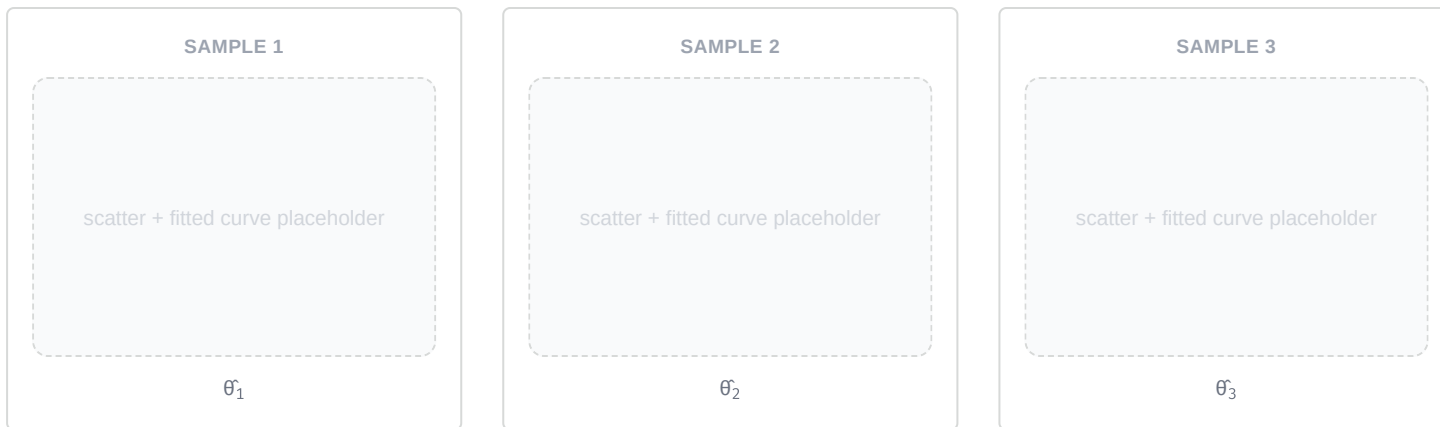
$$\mathcal{D} \sim p(x, y)$$

A finite collection of n samples
drawn i.i.d. from p

The dataset is a finite sample from an underlying distribution. The distribution does not change — but each sample will differ.

Different Samples, Different Models

Three datasets sampled from the same distribution → three different fitted models.



If we collected the data again, would we learn the same model?

What Do We Actually Want to Minimize?

WHAT WE OPTIMIZE

$$L_{\text{train}}(\theta)$$

Empirical risk — averaged over the finite training set $\mathcal{D}$

WHAT WE CARE ABOUT

$$L_{\mathcal{D}}(\theta) = \mathbb{E}_{p(x,y)}[\ell(f_{\theta}(x), y)]$$

Expected risk — performance on the full underlying
distribution

We optimize performance on the **sample**. We care about performance on the **distribution**.

Underfitting and Overfitting

UNDERFITTING

low-capacity model —
misses the structure

High bias. Cannot capture the pattern
even on training data.

GOOD FIT

appropriate capacity —
captures the structure

Generalizes. Fits the pattern without
fitting the noise.

OVERFITTING

high-capacity model —
fits sample noise

High variance. Excellent on training
data, poor on new data.

Bias and Variance

BIAS

Systematic error from a model that is **too restricted** to capture the true pattern.

→ Underfitting. Present even if trained on infinite data.

VARIANCE

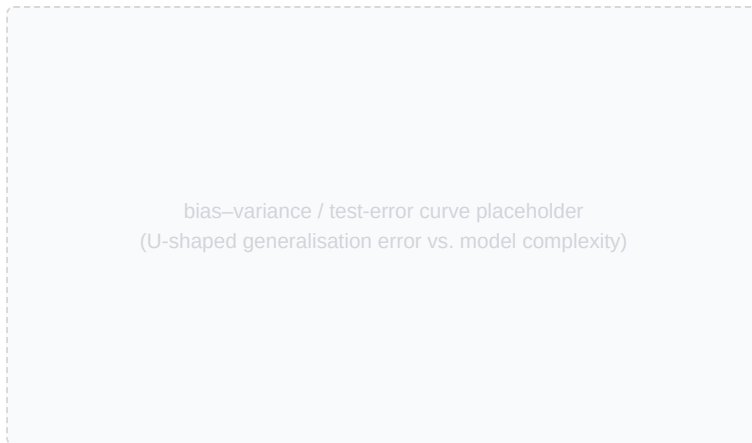
Sensitivity of the learned model to the **particular training sample**.

→ Overfitting. Different samples produce very different models.

A model with high bias always makes the same kind of mistake. A model with high variance makes different mistakes depending on which data it saw.

Model Capacity Changes the Trade-off

CLASSICAL ERROR CURVE



↑ Capacity → ↓ Bias

↑ Capacity → ↑ Variance (in the classical regime)

There is an intermediate complexity that generalizes best.

Classical intuition only — double descent will revise this picture in Block 2.

A Geometric View: Loss Surfaces

 Loss surface over parameter space with a basin of minimum training loss

RECAP FROM LECTURE 4

- Gradient descent navigates the loss surface
- The surface shape is determined by the training data
- Training selects parameters by finding a low point

The loss surface is induced by the **training data** — a different sample gives a different surface.

Different Data, Different Loss Surfaces

Three finite samples from the same distribution → three slightly different loss surfaces.



A solution that perfectly fits one sample may not be stable across other samples from the same distribution.

Regularization Changes Which Solution We Prefer

MANY LOW-LOSS SOLUTIONS EXIST

flat loss basin — many θ achieve
similarly low training loss

REGULARIZATION BIASES TOWARD SIMPLER SOLUTIONS

Smaller model (fewer parameters)

L2 / weight decay — penalizes large weights

Early stopping — halt before overfitting

Regularization is not only about reducing training loss —
it changes **which solution we prefer**.

Three Ways to Improve Generalization

RESTRICT THE MODEL

Use a smaller or simpler model.

- Fewer parameters
- Shallower network
- Lower model complexity

RESTRICT THE SOLUTION

Keep the model flexible — constrain what it learns.

- L2 weight decay
- Sparsity constraints
- Early stopping

INCREASE THE DATA

Train on more examples.

More data reduces dependence on the peculiarities of one finite sample — and allows larger models to be trained more reliably.

What happens if, instead of keeping the model small, we make **both the model and the dataset much larger**?

Block 2 — Generalization in the Overparameterized Regime

Classical intuition suggests that increasing capacity beyond the interpolation threshold should hurt generalization. Modern deep learning often behaves differently.

Block 2 — Coming Soon

Placeholder — content will be developed here.

Planned topics:

- Interpolation threshold
- Double descent
- Grokking
- Implicit regularization
- Overparameterization
- Scaling model capacity and data
- Connection to foundation models

Modern overparameterized networks can fit training data perfectly and still generalize. Why?

Block 3 — Beyond the Interpolation Threshold

Beyond the Interpolation Threshold

RECALL FROM BLOCK 1

Overparameterized polynomial placeholder —
wiggly curve interpolating all training points perfectly

A high-degree polynomial that perfectly fits the training set can behave pathologically between samples.

Interpolation threshold — the model capacity at which training loss can reach zero.

Why don't overparameterized neural networks necessarily behave like pathological high-degree polynomials?

The Classical Prediction Breaks Down

CLASSICAL EXPECTATION

- Increasing capacity reduces bias
- After some point, variance increases
- The test-error curve has a U-shape
- Beyond the apparent optimum, generalization deteriorates

U-shaped test-error curve placeholder (classical regime only)

But is this what we actually observe in modern deep learning?

Double Descent

CONCEPTUAL ERROR CURVE

train error + test error vs. model capacity

mark interpolation threshold

test error rises near threshold, then falls again

(double descent curve placeholder)

Train error reaches zero at the interpolation threshold.

Test error peaks near the threshold.

Then test error decreases again in the overparameterized regime.

The classical U-shaped curve can be incomplete.

Many Functions Can Fit the Training Data

Once the model is large enough to interpolate, many different functions all achieve zero training loss.

placeholder: several very different curves, all passing exactly through the same training points

$$\{ f \in \mathcal{H} : L_{\text{train}}(f) = 0 \}$$

If many functions fit the data perfectly, which one does learning select?

Parameter Space Is Not Function Space

PARAMETER SPACE

$$\theta \in \mathbb{R}^P$$

Many distinct weight vectors

FUNCTION SPACE

$$f_{\theta}(x)$$

The input-output mapping induced by θ

Many different θ may produce the same or very similar function. The mapping $\theta \rightarrow f_{\theta}$ is not one-to-one.

A large parameter space does not imply that every function is equally likely.

Overparameterization Does Not Mean Random Functions

INTUITION

- An enormous parameter space maps very unevenly into function space
- Some functions correspond to many different parameter configurations
- Simple or structured functions may be easier to reach from typical initializations

conceptual: many $\theta \rightarrow$ one simple function vs. few $\theta \rightarrow$ one complex function (placeholder)

More parameters increase what the model can represent, but do not imply uniform sampling over functions.

Inductive Bias Selects Among Interpolating Solutions

The hypothesis space may be huge, but learning has preferences.

Architecture

convolutional, recurrent, equivariant

Equivariances

translation, rotation, scale

Optimization

SGD, Adam, learning rate schedule

Initialization

affects which minimum is found

Regularization

explicit and implicit

...

Similarity and Smoothness in Learned Representations

WHAT THE NETWORK LEARNS

input space $\rightarrow$ representation space placeholder

similar inputs cluster together

- Inputs are mapped to a learned representation
- Nearby representations can lead to similar predictions
- The learned similarity structure shapes generalization

Generalization depends not only on fitting the samples, but on the geometry of the learned representation.

This prepares the transition to location encodings and continuous spatial functions (Lecture 6).

Grokking: Fitting Comes Before Generalizing

EMPIRICAL OBSERVATION

train accuracy vs. test accuracy over training steps placeholder —
train reaches ~100% early; test lags, then sharply improves much later

Training accuracy → near-perfect quickly

Test accuracy remains poor for much longer

Then test accuracy improves sharply

Memorization and discovering a reusable rule are not the same event.

What Do We Actually Know?

There is no single complete, universally accepted explanation for why highly overparameterized deep networks generalize so well.

CURRENT USEFUL PERSPECTIVES

Simplicity bias

Architectural inductive bias

Optimization bias

Representation geometry

Data scale

Explicit & implicit regularization

Modern generalization remains an active research topic.

From Generalization to Learned Spatial Functions

GENERIC LEARNED FUNCTION

$$f_{\theta}(x)$$



LOCATION-DEPENDENT FUNCTION

$$f_{\theta}(\text{location})$$

TOPICS IN LECTURE 6

- Location encodings
- Coordinate networks
- Interpolation between spatial observations
- Learned geospatial signals

What function does a neural network learn between observed locations?



LECTURE 5 (LEGACY)

DEEP REPRESENTATION LEARNING II

Learning beyond Memorization

PRESS – OR SPACE. →

© Marc Rußwurm

Licensed under [Creative Commons Attribution-NonCommercial 4.0 International \(CC BY-NC 4.0\)](#).

You may share and adapt these slides for teaching, research, and other non-commercial educational purposes with attribution.

Commercial training, paid workshops, consulting seminars, or incorporation into commercial course products require prior permission.

In this lecture (legacy)

- Neural networks and learned representations
- Supervised training and optimization
- Learning useful intermediate features directly from data
- Convolutional neural networks
- Geospatial and remote-sensing examples

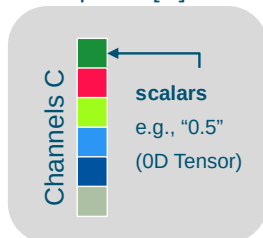
Data and Experience



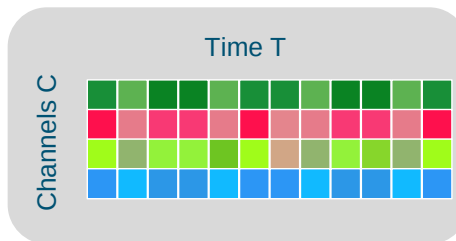
Data is the window through which a learner experiences the world.

Data as blocks of numbers - E.g., Tensors

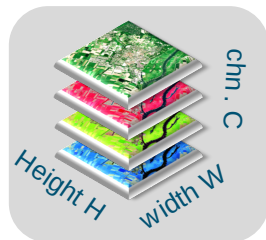
1D Tensor: Vector
e.g., pixels or
class probs [C]



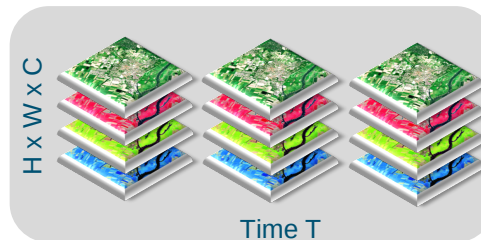
2D Tensor: Matrix
Time Series [T x C]



3D Tensors
Images [H x W x C]



4D Tensors
Img. Time Series [T x H x W x C]



Data samples from a distribution